

Esercizio 2 punti

Scelte algoritmiche e strutture dati

E' stata usata una matrice bidimensionale allocata dinamicamente, per ogni elemento della matrice precedente si è calcolato il numero di elementi ad esso adiacenti tramite la variabile k e da che indice partire e finire sia per righe che per colonne tramite $startn$ e $starts$, $finn$ e $fins$, se si è nella prima riga si toglie il controllo per la riga superiore inesistente, analogamente per l'ultima con la riga inferiore, modificato il numero di elementi e indice di inizio/fine in verticale si compie un'operazione analoga in orizzontale tenendo conto di evitare di togliere più volte stessi elementi da k . Trovato il range voluto, un doppio ciclo somma tutti gli elementi dentro a esso e divide per il numero di elementi in esso.

Differenze

- Aggiunta inizializzazione di t , aggiornato il nome da tmp a t anche dentro i calcoli, inizialmente si aveva una variabile tmp di tipo intero, è stato modificato la definizione della variabile e il nome di esso durante l'esame ma non si è aggiornato anche dentro la funzione.
- Modificato $<$ in $\leq$ in quanto si salvava l'ultimo indice dentro al range e non il primo in cui non va più bene.
- Spostata l'inizializzazione di k dentro al ciclo interno a un valore costante e reso costanti i valori sottratti a k , scambiato $r-1$ con $c-1$ in un for seguito da un altro, passaggio di matrice precedentemente come int nonostante la si fosse definita come $float$. In generale errori dovuti al bisogno di fare in fretta.
- Scambiato ordine ai lati dell'uguaglianza finale e invertito ordine di $malloc$ della nuova matrice all'inizio, si sarebbe dovuto prima salvare nel heap il vettore di puntatori e come secondo creare il vettore di interi tramite indice, usando $*NM$ invece si reiterava sullo stesso vettore in posizione zero. Errori dove si guarda ciò che si ha scritto e ci si chiede se lo si ha veramente scritto.

Esercizio 4 punti

Strutture dati e scelte algoritmiche

Albero contenente il numero di nodi n , non sapendo a priori il numero è stato usato 2 nel compito in quanto l'albero binario è la rappresentazione più comune. L'idea se n valesse due era di usare una funzione ricorsiva che aumentava di uno la lunghezza solo se il nodo visitato avesse valore positivo, il valore al ritorno nel caso terminale ($t==z$) viene diminuito di 1.

Durante l'esame si è risolto un sottoproblema specifico, non è stata letta con attenzione la traccia, se fosse n generico si utilizzerebbe un ciclo ma esso è stato implementato dopo la terminazione dell'esame ed è quindi invalido.

Differenze

- Aggiunta la parte di input e inizializzazione per permettere di testare il codice e il passaggio del nodo fittizio z come parametro nella funzione ricorsiva.
- Nome di funzione ricorsiva modificato da fu a $fuesame$ e tolto puntatori inutilizzati.

- Scambio tra vettore di puntatori ai figli e numero di figli dentro T e il nodo fittizio dentro il nodo, non aveva senso avere il vettore di figli nell'albero in quanto ogni nodo ha n figli mentre z poteva rimanere dentro al nodo (bastava passare $t \rightarrow \text{root} \rightarrow z$), quest'ultima è stata una scelta per una questione di modularità basandosi sui laboratori fatti.
- Modificato il valore passato a tmax in un ricorsione con nodo negativo, basta un solo nodo negativo per azzerare il conteggio, tenere il valore precedente è insensato.

Commenti

Come cammino si considera l'arco tra un nodo padre e il suo figlio, la funzione ritorna dunque il numero di archi passati, ovvero numero di nodi transitati -1 .

Esercizio 6 punti

Strutture dati e scelte algoritmiche

Si lavora con la sola matrice di adiacenza: sono state usate le disposizioni semplici in quanto ogni città è unica e non ci sono copie di esso prendibili più volte e quali città vengono inserite in un gruppo dipende dalle altre città già presenti nel gruppo stesso.

Nell'implementazione però ci sono state delle falle nel ragionamento e confusione tra quale variabile rappresentava cosa.

Infatti un modello corretto sarebbe usando le disposizioni ripetute ma invece di assegnare a ogni città un gruppo, ho più copie di un "gruppo" assegnabili a una città sempre seguendo le condizioni richieste.

Durante l'esame si cercava di disporre le città in un numero di gruppi crescenti. Concettualmente per ogni città se non è stata già presa si controlla se per ogni indice di elemento appartenente al gruppo (con start come numero di elementi attualmente nel gruppo), controllando la matrice di adiacenza, se sono presenti città non direttamente collegate si controlla che ci sia una città intermedia che sia collegato ad entrambe, se non lo si trova significa che non si può inserire l'elemento dentro al gruppo in costruzione quindi lo si mette in un nuovo gruppo di indice maggiore. Ma se il numero di gruppi supera il limite imposto si prova ordinando le città diversamente.

Nella ricorsione in terminazione se non sono stati presi tutti i valori, ovvero se almeno un mark risulta pari a -1 ritorna zero altrimenti la prima configurazione è quello che presenta il minor numero di gruppi, si torna quindi 1 che viene propagato fino al wrapper.

In primis gli indici che verrebbero usati nel controllo non dovrebbero essere tra 0 e il numero di elementi in quanto verrebbero controllati solo i primi i-esimi vertici della matrice ma solo quelli che appartengono realmente al gruppo (con $tm[i] == pos$). Per secondo la condizione di terminazione ($pos \geq k$) è errata in quanto la ricorsione termina solo quanto è stato trovato un elemento che fa superare il numero massimo di gruppi, ma quest'ultima non è modificabile senza modificare il modello di calcolo combinatorio usato o senza modificare il significato assegnato alle variabili utilizzate.

Differenze

- Aggiunte main, funzioni di libreria standard e funzioni dalla libreria ST per permettere di testare il codice
- Aggiunto * a tsol dentro control per permettere il corretto passaggio del vettore nella funzione e aggiunto il passaggio di mark, n e pos.
- Tolto il passaggio di *v e start e aggiunto il passaggio di N dentro findpath
- Aggiunto il controllo di appartenenza al gruppo attuale e di elemento selezionato, sostituito tsol[i] in matrice (v[i] dentro findpath) con il contatore i, e modifica della condizione di terminazione del ciclo for da <start a <N in quanto servono le coordinate degli elementi appartenenti al gruppo e non il gruppo a cui appartiene ogni elemento che ha indice minore di start.
- Aggiunta selezione di città dentro il ramo che fa aumentare il numero di gruppi
- Eliminato mark[i]=1 in posizione sbagliata che non è stato cancellato in tempo durante l'esame

Commenti

L'appartenenza della città intermedia che verrebbe attraversata in uno specifico gruppo è stato preso in considerazione nel pruning, in altre parole, la città intermedia deve appartenere allo stesso gruppo di quello che non ha il collegamento diretto con l'elemento che si sta per selezionare.

Nonostante il modello sia sbagliato è stato mantenuto anche successivamente per questioni di coerenza con il codice scritto in sede dell'esame.